

Draw It or Lose It
CS 230 Project Software Design Template
Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	11/14/2025	Reagan Morgan	Added Executive Summary, Requirements, and Design Constraints.
2.0	11/26/2025	Reagan Morgan	Added Evaluations of different operating systems.
3.0	12/14/2025	Reagan Morgan	Added Recommendations.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Our client, The Gaming Room, wants us to develop a web application version for their android app, Draw It or Lose It. They want us to help set up the environment for this application. They also want us to make sure our software can have multiple teams, with each team having multiple players assigned to each team, and they have unique team names & game names. They also want to make sure only one instance of the game can exist at any time. The following document outlines the solutions to our client's problems.

Requirements

1. Development of a web-based application. Apps must consider web service APIs and frameworks in Java, like Spring Boot and RESTful.
2. App must only be able to run once instance of a game at a time.
3. App must be able to support multiple teams with unique team names.
4. Each team should be able to support multiple players.
5. Each game instance must have a unique name.

Design Constraints

1. Web-based Application
    a. This constrains us to using frameworks like Spring Boot and RESTful API services.
2. Only one instance is allowed at a time.
    a. This constrains us to using a singleton design pattern for our application in order to ensure one instance is running at a time.
3. Multiple teams & Unique Names
    a. This constraint requires us to implement a Team class that stores each team and allows team creation that checks against currently stored teams to make sure that the team's name is unique.
4. Multiple players
    a. This constraint requires us to implement a Player class that creates and stores each player and assigns them to a team of their choice or randomly.
5. Unique game name
    a. This constraint requires us to Implement a check against the List of Games when creating a new game to make sure none of the game names are the same.

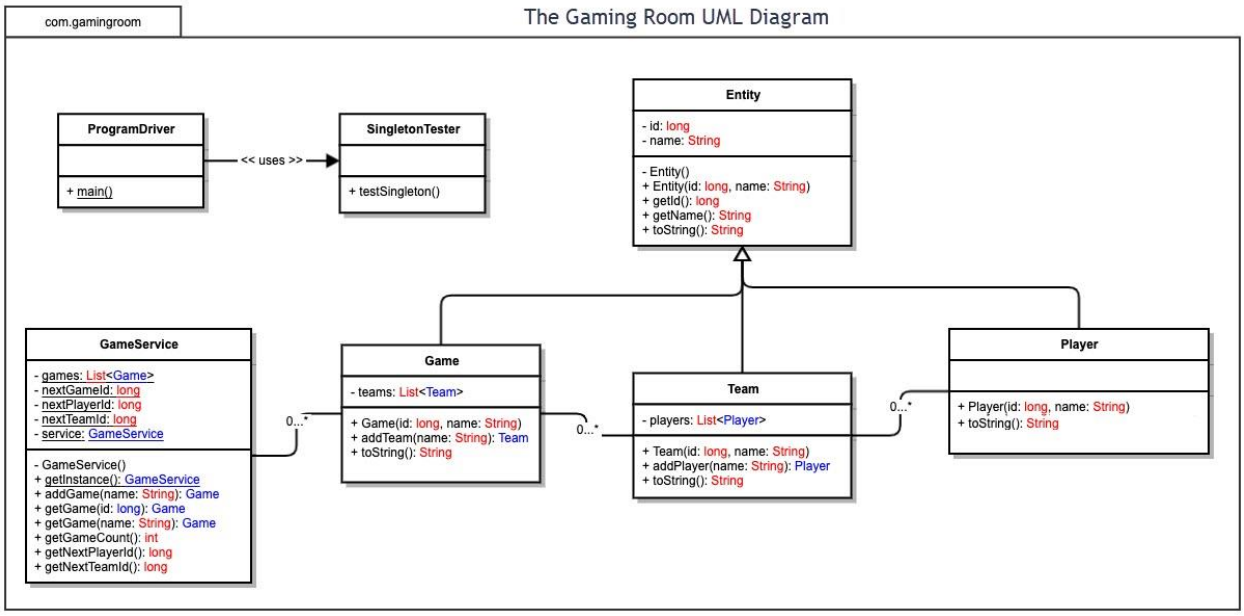
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

This UML diagram shows the class arrangement of the UML diagram for Draw It or Lose It. The Program Driver is used to test the program by creating new games and using the singleton driver to show the single instance of games. The Game Service class implements the singleton design pattern as well as adds and retrieves games. The game class is the data used for creating games. Each game needs to have

a list of teams as well as a way to add teams to the game. The team class is the data used for creating teams. Each team needs to have a list of teams as well as a method to add players to the team. The Player class holds the player's data. It also allows you to create a player. All three of these classes inherit the attributes and methods of the parent class, Entity. This class allows for the data from the 3 classes (Game, Team, Player) to be encapsulated by the getter methods used in the entity class. This also allows for the 3 classes to have enhanced readability and less redundancy by containing the duplicate attributes (id, name) and methods (getters) within the entity class.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	MacOS has the ability to host a web-based software application using its integrated server features. It is great for teams or companies who run on apple products. However, the price is steep and scalability is limited. There may also be issues once the version used stops receiving security updates due to being phased out.	Linux has the ability to host a web-based software application. Positives include free to low-cost options for servers, increased security, stability, less bloatware meaning more resources are used for what you need, and increased customization. Downsides include a learning curve and potential incompatibility with some commercial applications that only run on windows or macOS.	Windows has the ability to host a web-based server application. Positives include compatibility, user-friendly interface, and integration with microsoft tools. Downsides include higher cost (although not as high as macOS), and limited virtualization.	Mobile devices have the ability to host a web-based server application. Some positives include portability, cost-effectiveness, and local storage. Downsides include decreased security, inability to process large amounts of traffic, limited processing power, and limited power and battery life.

Client Side	MacOS is very easy to understand for beginners, making the learning process not as tedious as Linux or Windows. However, macOS has no ability to accessed outside of other mac devices, which could run a steep cost and limit development time if the company does not have enough money to supply developer with Mac devices as well as give those who have no experience developing on a mac time to learn. This could limit performance and flow for developers and increase upfront costs for The Gaming Room.	Linux is extremely affordable, which is a pro for The Gaming Room who wants to keep their project within budget. However, Linux also comes with a steep learning curve in most cases. Developers will need to learn how to develop from the terminal or at least know how to use the terminal within Linux. Developers will also need to know the basics of how Linux and its security protocols work as well as be able to troubleshoot and look up solutions to potential OS problems on their own as there is no tech support services like there are for MacOS or Windows.	Windows is very middle of the road in terms of difficulty in understanding. Developers should not take too long to learn and understand the basics behind using windows. However, while it is lower in cost compared to macOS, there are still cost considerations to be made as some features may cost extra when looking at different versions of Windows.	Development can happen with mobile devices, however it is limited in what can be achieved and the efficiency of development. Cost is fairly cheap, but there are a lot of trade-offs for that cheap price when it comes to time spent on development.

Development Tools	Programming Languages: -PHP -Python -Java -JavaScript -Ruby -C# -Swift IDEs/Tools: -VSCode -JetBrains (different apps tailored to specific languages) -XCode (for Apple ecosystem development; only available on macOS) -Android Studio (for Android development) -Eclipse -Apache NetBeans	Programming Languages: -PHP -Python -Java -JavaScript -Ruby -C# -Swift IDEs/Tools: -VSCode -JetBrains (different apps tailored to specific languages) -Android Studio (for Android development) -Eclipse -Apache NetBeans	Programming Languages: -PHP -Python -Java -JavaScript -Ruby -C# -Swift IDEs/Tools: -VSCode -JetBrains (different apps tailored to specific languages) -Android Studio (for Android development) -Eclipse -Apache NetBeans	Programming Languages: -Python -Java -JavaScript -C# -Swift -Kotlin IDEs/Tools: -Pocket Editor -Dcoder

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: I would recommend Linux, specifically the Ubuntu distribution. It's cheap, fast, secure, and reliable.

2. **Operating Systems Architectures**: Ubuntu has similar architecture to other operating systems, meaning it has 4 main layers: Hardware, Kernel, Shell, and Software. The hardware layer manages all hardware components like CPU, RAM, Storage devices, I/O devices, etc. The kernel, or operating system layer, is what interfaces between the shell and hardware layers, making it responsible for all hardware operations like memory management, I/O device interactions, allocating resources, etc. The Shell is what interfaces and interprets commands from the software layer to the kernel layer. The software layer encompasses everything that users interact with. This could also be considered as the user layer.

3. **Storage Management**: For the Gaming Room, I recommend using a cloud based storage management system like Amazon Web Service's Simple Storage Service, Google Cloud Storage, or Azure Storage. This will allow them to stay within their initial budget while also allowing for cheap scalability as the game grows.

4. **Memory Management**: Linux divides RAM into 4 distinct sections: Physical memory, virtual memory, kernel space, and user space. For the purposes if this, we will be focusing on physical and virtual memory. Virtual memory essentially divides itself and the physical memory into pages. The kernel then creates a page table where the virtual memory addresses are mapped onto physical memory addresses. So instead of an application of program access the physical memory address, it receives the virtual memory address. This allows for better data security and only loads in necessary parts of a program into the memory. Another important memory management technique Linux uses is having a swap space. The swap space acts as an extension of RAM. It stores pages of memory that are not frequently used to free up memory for more important tasks the user is currently working on. For memory allocation, Linux uses dynamic memory allocation, memory overcommit, and page caching. Since memory is so important, I recommend The Gaming Room also regularly run vmstat 1 during deployment and testing to see how the memory is being used.

5. **Distributed Systems and Networks**: It is recommended that The Gaming Room use a client-server architecture, using the previously recommended ubuntu server to handle all backend game functions (logic, matchmaking, sync across clients) while the client connects to the server via WebSocket. I recommend The Gaming Room connect using WebSocket instead of REST APIs since we want our data transfers from server to client to be in real-time. To manage connectivity issues and server outages, I recommend The Gaming Room implement a cloud-based version of their server as well. This will allow them to not only balance the load of incoming network traffic by having more than one server but also allow the game to keep running if the area the client operates out of had a power outage or internet outage.

6. **Security**: Ubuntu is created to have security features as soon as you install the distribution. It comes with an active firewall, kernel livepatch which helps reduce downtime by allowing security updates to be implemented without shutting down, and role-based access control which limits what other users have permission to view or access. Since potentially sensitive user information may be stored for user account purposes, The Gaming Room should make sure the database and its data is encrypted, only accessible to certain roles using the principle of least privilege, regularly backed up, and frequent updates of the database management system software.